

Algoritmer [CCT3, COMTEK3, & ESD3]
RPS 1: Review Exercise and Programming Session 1 **09/2024**

Introduction: The exercises in this session are designed to cover the main concepts from the first part of the course which focuses on basic algorithms and concepts. The exercises cover the main concepts from Modules 1 – 5. You are expected to post the running time estimate of each of the algorithms implemented during the exercise.

Exercise 1: Analysis, Design and Implementation of Search Algorithms

Consider the following algorithms for solving the search problem, i.e., finding an entry, v in an array, A .

```
1: procedure L-SEARCH( $A, n, v$ )
2:   for  $i = 0$  to  $n - 1$  do
3:     if  $A[i] \geq v$  then
4:       return  $i$ 
5:     end if
6:   return NoTFound
7: end for
8: end procedure

1: procedure B-SEARCH( $A, l, r, v$ )
2:   while  $l \leq r$  do
3:      $m = \lfloor (l + r) / 2 \rfloor$ 
4:     if  $A[m] == v$  then
5:       return  $m / (m - 1)$ 
6:     else if  $A[m] < v$  then
7:        $l = m + 1$ 
8:     else
9:        $r = m - 1$ 
10:    end if
11:  end while
12:  return NoTFound
13: end procedure

1: procedure 4-SEARCH( $A, l, r, v$ )
2:   if  $l > r$  then
3:     return NoTFound
4:   end if
5:    $m_1 = \lfloor l + (r - l) / 4 \rfloor$ 
6:    $m_2 = \lfloor l + 2 \times (r - l) / 4 \rfloor$ 
7:    $m_3 = \lfloor l + 3 \times (r - l) / 4 \rfloor$ 
8:   if  $A[m_1] == v$  then
9:     return  $m_1$ 
10:  end if
11:  if  $A[m_2] == v$  then
12:    return  $m_2$ 
13:  end if
```

```

14:   if  $A[m_3] == v$  then
15:       return  $m_3$ 
16:   end if
17:   if  $v < A[m_1]$  then
18:       return 4-SEARCH( $A, l, m_1 - 1, v$ )
19:   else if  $v < A[m_2]$  then
20:       return 4-SEARCH( $A, m_1 + 1, m_2 - 1, v$ )
21:   else if  $v < A[m_3]$  then
22:       return 4-SEARCH( $A, m_2 + 1, m_3 - 1, v$ )
23:   else
24:       return 4-SEARCH( $A, m_3 + 1, r, v$ )
25:   end if
26: end procedure

```

- Complete the following table about the characteristics, design framework and implementation form of algorithms - L-SEARCH, B-SEARCH and 4-SEARCH.
- Based on your answer from (a) above, determine which of the algorithms need to be modified and correct it such that it returns the correct output for all valid input.
- Find an expression for the running time of each of the algorithms as a function of the input size, n . *Hint: If the algorithm is iterative, use statement-wise cost analysis, otherwise write the recurrence relation.*
- Using the expressions in (c) above and any suitable method (from those covered during the lectures), determine the asymptotic complexity of each of the algorithms.
- In any programming language of your choice, implement the three search algorithms and verify that your implementations work correctly.
- Using an array of uniformly distributed between 1 and 100 with length $n = 10^7$, estimate the running time of the search algorithms averaged over 100 repeated runs each with different randomly generated integer array. Using the search value, $v = 105$.

Table 1: Characteristics, Design Framework and Implementation of Search Algorithms

Algorithm	Correct	Efficient	Effective	Definite	Finite	Implementation	Design Approach
L-SEARCH							
B-SEARCH							
4-SEARCH							

Exercise 2: Analysis, Design and Implementation of Sorting Algorithms

Consider the following algorithm that sorts an integer array, A . The algorithm uses the procedure MERGE to merge four subarrays in sorted order. Procedure MERGE combines the four subarrays in pairs. It first merges A_1 with A_2 and A_3 with A_4 . The results are then merged to obtain the final sorted array. The MERGE procedure has a complexity of $O(n)$.

```

1: procedure Q-MERGESORT( $A$ )
2:    $n = \text{length}(A)$ 
3:   if  $n == 1$  then

```

```

4:     return A
5: end if
6:    $q_1 = \lfloor n/4 \rfloor$ 
7:    $q_2 = \lfloor n/2 \rfloor$ 
8:    $q_3 = \lfloor 3n/4 \rfloor$ 
9:    $A_1 = A[1 : q_1]$ 
10:   $A_2 = A[q_1 + 1 : q_2]$ 
11:   $A_3 = A[q_2 + 1 : q_3]$ 
12:   $A_4 = A[q_3 + 1 : n]$ 
13:   $A_1 = \text{Q-MERGESORT}(A_1)$ 
14:   $A_2 = \text{Q-MERGESORT}(A_2)$ 
15:   $A_3 = \text{Q-MERGESORT}(A_3)$ 
16:   $A_4 = \text{Q-MERGESORT}(A_4)$ 
17:  return MERGE( $A_1, A_2, A_3, A_4$ )
18: end procedure

```

(a) Which of the following statements about algorithm Q-MERGESORT is(are) true? There may be multiple correct statements. Tick all that apply.

- A. Q-MERGESORT uses the divide-and-conquer principle and its implementation is recursive.
- B. Q-MERGESORT uses the divide-and-conquer principle and its implementation is iterative.
- C. Q-MERGESORT uses the greedy technique and its implementation is recursive.
- D. Q-MERGESORT uses brute-force technique and its implementation is iterative.
- E. Q-MERGESORT uses brute-force technique and its implementation is iterative.

(b) Illustrate the operation of Q-MERGESORT algorithm with the following input:

$$A = [8, 4, 6, 9, 2, 7, 1, 5, 14, 11, 15, 3, 12, 13, 10, 16]$$

- (c) Write the recurrence relation for Q-MERGESORT as a function of the input size, n .
- (d) Using the master method, determine the solution of the recurrence relation in (c).
- (e) Using any programming language of your choice, implement algorithm Q-MERGESORT and verify that it works correctly.
- (f) Randomly generate an array, A containing $n = 10^8$ integers with values uniformly distributed between 1 and 100. Apply Q-MERGESORT and your implementation of the standard MERGESORT from the lectures on A and record the running time. Repeat the data generation and sorting 100 times and compute an average of the running time of each algorithm.

Exercise 3: Analysis, Design and Implementation of Greedy and Dynamic Programming Algorithms

Consider the problem of making change for n cents using the smallest number of coins. Assume that each coin's value is an integer.

- (a) Describe a greedy algorithm to make change consisting of quarters (25 cents), dimes (10 cents), nickels (5 cents), and pennies (1 cents). *Note: It is sufficient to describe the steps of the algorithm in plain text. You do not need to give a pseudocode.*
- (b) Give a set of coin denominations for which the greedy algorithm does not yield an optimal solution. Your set should include a penny so that there is a solution for every value of n .
- (c) Give an $O(nk)$ -time algorithm that makes change for any set of k different coin denominations using the smallest number of coins, assuming that one of the coins is a penny.
- (d) Implement the greedy algorithm in (c) and verify that it works.

RAD